

Iskolai Raktár

Vizsgaremek dokumentáció

Telepítés, indítás és frontend dokumentáció

Tartalomjegyzék

1. A program használata GitHubról	3
1.1 Szükséges programok	3
1.2 Projekt letöltése	3
1.3 Adatbázis-beállítás	3
1.4 A program indítása	3
1.5 Első bejelentkezés	4
1.6 Első beállítások	4
1.7 QR-kód alapú leltározás	4
2. Frontend dokumentáció	5
2.1 A frontend feladata	5
2.2 Alkalmazott technológiák	5
2.3 Razor Pages felépítés	5
2.4 Bejelentkezési oldal	5
2.5 Kezdőlap	6
2.6 Leltárkezelő felület	6
2.7 Adminisztrációs felület	7
2.8 Jogosultságkezelés	7
2.9 Leltárkörzetek kezelése	7
2.10 QR-kód beállítások	8
2.11 QR-kódos leltározás	8
2.12 JavaScript architektúra	9
2.13 Service réteg	9
2.14 ApiService	9
2.15 Backend API elérése	10
2.16 Munkamenet-kezelés	10
2.17 Hibakezelés és felhasználói visszajelzés	11
2.18 A frontend felépítésének előnyei	11
2.19 Összefoglalás	11

Backend dokumentáció.....	13
3. Rendszer áttekintés és Architektúra.....	13
4. Projektstruktúra és Összetevők.....	13
5. Konfigurációs Fájlok és Környezeti Beállítások.....	14
5.1 "appsettings.json"	15
5.2 "dbsettings.json"	15
5.3 "certsettings.json"	15
5.4 "qrsettings.json"	15
5.5 "auth.json" (Automatikusan jön létre).....	15
6. Kezdeti Inicializáció és Indítási Folyamat.....	15
7. Dinamikus Tábla- és Adatkezelő Motor.....	16
7.1 A dinamikus architektúra lényege.....	16
7.2 Rendszer által kezelt fix (fenntartott) oszlopok.....	16
7.3 Biztonság és SQL Injection elleni védelem.....	16
8. Authentikáció és Jogosultságkezelés.....	17
8.1 Felhasználói modell és állapot.....	17
8.2 Jogosultsági szintek ("AccessLevel").....	17
8.3 Hozzáférés-szabályozási szabályok.....	17
9. QR-Kód és Leltározó Rendszer.....	17
9.1 QR-kód generálása ([iskolaraktarBackend/Controllers/QrController.cs] (iskolaraktarBackend/Controllers/QrController.cs)).....	18
9.2 Leltár szkennelése ([iskolaraktarBackend/Controllers/ScanController.cs] (iskolaraktarBackend/Controllers/ScanController.cs)).....	18
10. Részletes API Végpont Referencia.....	18
11. Telepítés, Publikálás és Üzemeltetés.....	18
11.1 Önálló (Self-Contained) Publikálás.....	18
11.2 HTTPS és Tanúsítvány Kezelés.....	19
12. Fejlesztési és Biztonsági Útmutató.....	19

1. A program használata GitHubról

A projekt GitHubról letölthető vagy klónozzható, majd Visual Studio segítségével futtatható.

1.1 Szükséges programok

A futtatáshoz az alábbiak szükségesek:

- Visual Studio ASP.NET/.NET fejlesztési támogatással
- .NET 10 SDK
- MySQL Server 8
- Git vagy a Visual Studio beépített Git támogatása

1.2 Projekt letöltése

A repository klónozzható Git segítségével:

```
git clone https://github.com/brudergyuri/iskolaraktar.git
```

Ezután meg kell nyitni az alábbi solution fájlt Visual Studio-ban:

```
iskolaraktarBackend.sln
```

1.3 Adatbázis-beállítás

A backend a MySQL kapcsolati adatokat az alábbi fájlból olvassa:

```
iskolaraktarBackend/dbsettings.json
```

A fájlban a helyileg telepített MySQL Serverhez tartozó felhasználónevet és jelszót kell megadni. Ezek az adatbázis-szerverhez tartozó hitelesítési adatok, nem az Iskolai Raktár alkalmazásba történő bejelentkezés adatai.

Az iskolaraktar adatbázist nem szükséges kézzel létrehozni. A backend első indításkor ellenőrzi, hogy létezik-e az adatbázis, és ha nem, automatikusan létrehozza.

1.4 A program indítása

A projekt két részből áll:

Projekt	Feladat
iskolaraktarBackend	ASP.NET Core REST API és MySQL adatkezelés
iskolaraktarFrontend	ASP.NET Core Razor Pages felhasználói felület

A rendszer megfelelő működéséhez mindkét projektnek egyszerre kell futnia.

Visual Studio-ban ehhez a solution indítási beállításainál a Multiple startup projects lehetőséget kell kiválasztani, majd az iskolaraktarBackend és az iskolaraktarFrontend projektnél is a Start műveletet kell beállítani.

Ezután a Visual Studio Start gombjával vagy az F5 billentyűvel a backend és a frontend egyszerre indítható el.

A frontend ezután automatikusan a futó backend API-val kommunikál a saját /api/... útvonalain keresztül.

1.5 Első bejelentkezés

Ha még nem létezik auth.json, a program első indításkor automatikusan létrehoz egy adminisztrátori felhasználót:

```
Felhasználónév: admin  
Jelszó: admin123
```

A meglévő auth.json fájlt a program nem írja felül.

1.6 Első beállítások

Bejelentkezés után az Adminisztráció felületen lehet:

- új felhasználókat létrehozni,
- felhasználói jogosultságokat beállítani,
- leltárkörzeteket létrehozni és törölni,
- a QR-kódokhoz használt szervercímet beállítani.

Új telepítés esetén a leltárkörzeteket nem szükséges előre létrehozni az adatbázisban, azok az adminisztrációs felületen hozhatók létre.

1.7 QR-kód alapú leltározás

A rendszer támogatja a QR-kód alapú leltározást. Minden leltári eszköz egyedi QR-azonosítóval rendelkezik, amelyből a rendszer QR-kódot generál.

A QR-kód beolvasásakor a rendszer megnyitja az adott eszközhöz tartozó leltározási oldalt, azonosítja az eszközt, majd frissíti annak utolsó leltározási időpontját.

A QR-kód telefonról vagy más eszközről történő használatához a frontendnek hálózaton keresztül elérhető címmel kell rendelkeznie. Ez például saját szerverrel vagy tárhellyel, fejlesztési és bemutatási környezetben pedig Visual Studio Dev Tunnel használatával oldható meg.

A publikus frontend címet az adminisztrációs felületen, a QR-kód beállítások részénél lehet megadni. A rendszer ez alapján készíti el a QR-kódokban szereplő hivatkozást.

Helyi localhost cím esetén a QR-kód a számítógépen működhet, de egy telefonról nem érhető el közvetlenül, ezért ilyen használatához publikus vagy helyi hálózaton elérhető cím szükséges.

2. Frontend dokumentáció

2.1 A frontend feladata

Az iskolaraktárFrontend projekt biztosítja az Iskolai Raktár rendszer grafikus felhasználói felületét. A frontend segítségével a felhasználók bejelentkezhetnek a rendszerbe, megtekinthetik a számukra elérhető leltárkörzeteket, kezelhetik a leltári eszközöket, valamint megfelelő jogosultság esetén adminisztrációs műveleteket is végezhetnek.

A frontend nem közvetlenül kezeli az adatbázist. Minden adatkezelési művelet a backend REST API-n keresztül történik. Így az adatbázis-kezelés, az üzleti logika és a megjelenítés egymástól elkülönül.

A frontend ASP.NET Core Razor Pages alapú alkalmazás, amely HTML-, CSS-, Bootstrap- és JavaScript-elemeket használ.

2.2 Alkalmazott technológiák

A frontend főbb technológiái:

- ASP.NET Core Razor Pages – az oldalak és a szerveroldali frontend működés alapja
- HTML5 – az oldalak szerkezete
- CSS – saját megjelenési beállítások
- Bootstrap – reszponzív felület, űrlapok, gombok, táblázatok és egyéb komponensek
- JavaScript ES Modules – a kliensoldali működés
- Fetch API – kommunikáció a backend REST API-val
- sessionStorage – az aktuálisan bejelentkezett felhasználó adatainak tárolása a böngésző munkamenete alatt

A JavaScript-kód modulárisan van felépítve, így az egyes funkciók külön fájlokban találhatók.

2.3 Razor Pages felépítés

A frontend ASP.NET Core Razor Pages technológiát használ.

Egy Razor oldal jellemzően két részből áll. A .cshtml fájl tartalmazza a megjelenítendő HTML/Razor felületet, míg a hozzá tartozó .cshtml.cs fájl a szerveroldali PageModel osztályt.

Például:

```
Pages/  
    Scan.cshtml  
    Scan.cshtml.cs
```

A Razor Pages előnye, hogy az egyes oldalakhoz tartozó megjelenítés és szerveroldali logika jól elkülöníthető.

A projektben ugyanakkor a felhasználói interakciók jelentős részét JavaScript kezeli. A Razor Pages elsősorban az oldalak kiszolgálását és szerkezetét biztosítja, míg az adatok betöltése és módosítása REST API-hívásokkal történik.

2.4 Bejelentkezési oldal

A Login oldal feladata a felhasználó azonosítása.

A felhasználó megadja:

- felhasználónév

- jelszó

A login-controller.js átveszi az űrlap adatait, majd az AuthService segítségével elküldi azokat a backendnek.

A kérés a következő API-végpontra kerül:

```
POST /api/auth/login
```

Sikeres bejelentkezés után a backend visszaadja a felhasználó adatait és jogosultságait. Ezek a böngésző sessionStorage tárhelyére kerülnek.

Például:

```
sessionStorage.setItem(  
    "currentUser",  
    JSON.stringify(user)  
);
```

Ezután a felhasználó a kezdőlapra kerül.

2.5 Kezdőlap

A kezdőlap biztosítja a rendszer fő funkcióinak elérését.

Innen érhető el többek között:

- Leltárkörzetek
- Adminisztráció

A felület célja, hogy a felhasználó a bejelentkezés után egyszerűen elérje a számára szükséges funkciókat.

Az aktuálisan bejelentkezett felhasználó adatai a közös oldalelrendezésben is megjeleníthetők.

2.6 Leltárkezelő felület

A leltárkezelő oldal jeleníti meg a rendszerben létrehozott leltárkörzeteket.

Például ilyen leltárkörzet lehet:

- informatika
- kémia
- testnevelés

A leltárkörzetek valójában a backend által kezelt dinamikus adatbázis-táblák.

A felhasználó csak azokhoz a leltárkörzetekhez fér hozzá, amelyekhez megfelelő jogosultsággal rendelkezik.

Egy leltári tétel többek között az alábbi adatokat tartalmazhatja:

- Eszközzazonosító
- Név
- Hely
- Leírás
- QR-azonosító
- Utolsó leltározás időpontja

Az oldal az InventoryService segítségével tölti le az adatokat a backend API-ból.

Az adatok kezelése a felhasználó jogosultságától függ.

Read jogosultság esetén az adatok megtekinthetők.

ReadWrite jogosultság esetén az adatok módosíthatók is.

Az adminisztrátor minden leltárkörzethez teljes hozzáféréssel rendelkezik.

2.7 Adminisztrációs felület

Az Admin oldal kizárólag adminisztrátori felhasználók számára használható.

Az oldal betöltésekor az AdminController ellenőrzi az aktuális felhasználót.

Ha nincs bejelentkezett felhasználó, a rendszer visszairányítja a Login oldalra.

Ha a felhasználó nem adminisztrátor, akkor nem használhatja az adminisztrációs felületet.

Az adminisztrációs oldalon több külön funkció érhető el.

Felhasználók kezelése

Az adminisztrátor hozhat létre új felhasználót.

Megadható:

- felhasználónév
- jelszó
- adminisztrátori jogosultság

A frontend az adatokat a backendhez továbbítja, ahol a felhasználó létrehozása és a jelszó hash-elése történik.

A meglévő felhasználók törölhetők is.

A rendszer megakadályozza, hogy az aktuálisan bejelentkezett adminisztrátor saját magát a felületről törölje.

2.8 Jogosultságkezelés

A nem adminisztrátori felhasználókhöz leltárkörzetenként külön jogosultság rendelhető.

A használható jogosultsági szintek:

- None
- Read
- ReadWrite

A frontend ezeket magyar nyelven jeleníti meg:

- Nincs hozzáférés
- Megtekintés
- Módosítás

A jogosultság megváltoztatásakor az AdminController az AdminService segítségével elküldi a módosítást a backendnek.

Ennek köszönhetően például egy felhasználó rendelkezhet olvasási jogosultsággal az informatika leltárkörzethez, miközben egy másik leltárkörzethez egyáltalán nincs hozzáférése.

2.9 Leltárkörzetek kezelése

Az adminisztrátor új leltárkörzeteket is létrehozhat.

A frontend elküldi a megadott leltárkörzet nevét a backendnek, amely létrehozza a hozzá tartozó adatbázis-táblát.

Ennek köszönhetően a rendszerhez nem szükséges előre rögzíteni minden használható leltárkörzetet.

Új telepítés esetén az adatbázis akár üresen is indulhat, majd az adminisztrátor a frontend segítségével létrehozhatja a szükséges területeket.

A leltárkörzet törölhető is. Törlés előtt a frontend megerősítést kér a felhasználtól, mivel a művelet a leltárkörzetben tárolt adatok törlésével jár.

2.10 QR-kód beállítások

Az adminisztrációs felületen megadható a QR-kódok által használt frontend alapcím.

Például:

```
https://szerver.raktar.com
```

vagy fejlesztési környezetben:

```
https://valamilyen-cim.devtunnels.ms
```

Ez azért szükséges, mert a generált QR-kódnak olyan címet kell tartalmaznia, amely a QR-kódot beolvasó eszközről is elérhető.

A localhost cím egy másik eszközről, például mobiltelefonról nem használható közvetlenül.

A bemutatási környezetben erre Visual Studio Dev Tunnel használható, éles rendszer esetén pedig saját szerver vagy más publikus tárhely biztosíthatja az alkalmazás elérését.

2.11 QR-kódos leltározás

A leltári eszközökhöz a rendszer egyedi QrGuid azonosítót rendel.

A frontend lehetőséget biztosít az adott eszköz QR-kódjának megjelenítésére.

A QR-kód egy olyan hivatkozást tartalmaz, amely például a következő formájú:

```
https://szerver-cime/scan/egyedi-qr-azonosito
```

A QR-kód mobiltelefonnal történő beolvasásakor a frontend Scan oldala nyílik meg.

Az útvonal:

```
/scan/{qrGuid}
```

A Scan.cshtml.cs a URL-ből átveszi a QR-azonosítót.

A kliensoldali scan-controller.js ezután elküldi azt a backendnek:

```
POST /api/scan/{qrGuid}
```

A backend megkeresi, hogy melyik leltárkörzetben található az adott azonosítóhoz tartozó eszköz.

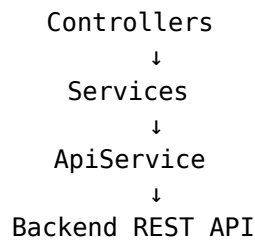
Sikeres találat esetén frissíti az eszköz LastInventoryDate értékét, majd visszaküldi az eszköz adatait a frontendnek.

A frontend ezután visszajelzést jelenít meg a sikeres leltározásról.

2.12 JavaScript architektúra

A JavaScript kód több rétegre van bontva.

A legfontosabb rétegek:



Controller réteg

A controller osztályok az adott oldal működéséért felelnek.

Feladatuk például:

- HTML elemek lekérése
- eseménykezelők létrehozása
- űrlapadatok feldolgozása
- service metódusok meghívása
- eredmények megjelenítése
- hibaüzenetek kezelése

Ilyen controller például:

- login-controller.js
- admin-controller.js
- inventory-controller.js
- scan-controller.js

A controller tehát nem közvetlenül az adatbázissal kommunikál, hanem service osztályokat használ.

2.13 Service réteg

A service osztályok feladata a backend API-val történő kommunikáció.

Például:

- AuthService
- AdminService
- InventoryService

Az AuthService a bejelentkezéssel kapcsolatos API-hívásokat kezeli.

Az AdminService többek között a felhasználók, jogosultságok, leltárkörzetek és QR-beállítások kezeléséhez szükséges API-kat használja.

Az InventoryService a leltári adatok lekéréséért és módosításáért felel.

Ez a felépítés azért előnyös, mert a hálózati kommunikáció nincs közvetlenül összekeverve az oldal megjelenítését végző kóddal.

2.14 ApiService

Az API-kommunikáció közös alapját az ApiService.js biztosítja.

Ez tartalmazza az általános HTTP-műveleteket:

- GET
- POST
- PUT
- DELETE

A többi service erre épül, így nem szükséges minden egyes funkciónál újra megírni a fetch() hívások teljes kezelését.

Például egy POST kérésnél az ApiService JSON-formátumba alakítja az adatot, és megfelelő HTTP-metódussal küldi el a backendnek.

Ez csökkenti a kódismétlést és átláthatóbbá teszi a frontend felépítését.

2.15 Backend API elérése

A frontend JavaScriptben nincs szükség fix publikus backend cím megadására.

A kérések relatív útvonalakat használnak:

```
/api/auth/login  
/api/auth/users  
/api/tables  
/api/data/...  
/api/scan/...
```

A frontend ASP.NET Core alkalmazás a /api kezdetű kéréseket továbbítja a helyben futó backendnek.

A frontend Program.cs-ében ezért egy HTTP kliens található, amelynek helyi backend címe:

```
http://localhost:5136
```

A működés:

```
Böngésző → Frontend → /api/... → Frontend proxy → Backend → MySQL
```

Ennek fontos előnye, hogy a JavaScript-kódban nem kell külön címet használni attól függően, hogy az alkalmazás localhoston vagy Dev Tunnelen keresztül fut.

Például a telefon a publikus frontend címet éri el, de az API-kérést továbbra is maga a frontend szerver továbbítja a helyi backendnek.

2.16 Munkamenet-kezelés

A projekt SessionService osztályt használ az aktuális felhasználó adatainak kezelésére.

A felhasználó sikeres belépése után az adatok a böngésző sessionStorage tárhelyére kerülnek.

A sessionStorage tartalma az adott böngészőfűl munkamenetéhez kapcsolódik.

A frontend ebből tudja meghatározni többek között:

- ki van bejelentkezve
- adminisztrátor-e
- milyen leltárkörzetekhez férhet hozzá

Kijelentkezéskor az aktuális felhasználói adatok törölhetők a munkamenetből.

2.17 Hibakezelés és felhasználói visszajelzés

A frontend az API-műveletek eredményéről visszajelzést ad.

Sikeres művelet esetén például megjelenhet:

A felhasználó sikeresen létrejött.

Hiba esetén a backend vagy a frontend által előállított hibaüzenet jelenik meg.

Az aszinkron műveletek try-catch blokkok segítségével vannak kezelve.

Ez lehetővé teszi, hogy hálózati vagy backendhiba esetén az alkalmazás ne álljon le, hanem érthető hibaüzenetet jelenítsen meg.

2.18 A frontend felépítésének előnyei

A frontend kialakításánál fontos szempont volt a funkciók elkülönítése.

A Razor Pages biztosítja az oldalak alapját, a controller JavaScript fájlok kezelik az oldalak működését, a service osztályok az API-kommunikációt végzik, míg az ApiService biztosítja az általános HTTP-kezelést.

Ennek eredményeként a felület később könnyebben továbbfejleszthető, mivel az egyes funkciók külön fájlokban és külön felelősségi körökben helyezkednek el.

2.19 Összefoglalás

Az Iskolai Raktár frontend egy ASP.NET Core Razor Pages alapú webalkalmazás, amely moduláris JavaScript segítségével kommunikál a backend REST API-val.

A frontend biztosítja a bejelentkezést, a leltárkörzetek és leltári tételek megjelenítését és kezelését, az adminisztrációs funkciókat, a felhasználói jogosultságok kezelését, valamint a QR-kód alapú leltározás felületét.

A frontend és backend közötti proxy használata lehetővé teszi, hogy ugyanaz a kliensoldali kód helyi és publikus, például Dev Tunnel alapú környezetben is használható legyen.

Backend dokumentáció

3. Rendszer áttekintés és Architektúra

Az **Iskolaraktár Backend** egy .NET 10.0 alapon nyújtott, ASP.NET Core Web API keretrendszerre épülő raktárkezelő és leltározó szerveralkalmazás. Az alkalmazást úgy terveztük meg, hogy rugalmasan alkalmazkodjon a különböző iskolák és intézmények egyedi raktározási igényeihez, ezért **séma-agnosztikus, dinamikus adatbázis-kezelő motorral** rendelkezik.

Főbb rendszerfunkciók

- **Dinamikus tábla- és oszlopkezelés:** Futásidőben hozhatók létre új raktári kategóriák (táblák) és egyedi mezők (oszlopok) kódmódosítás vagy fordítás nélkül.
- **Automatikus leltári alapszerkezet:** Minden dinamikus tábla automatikusan tartalmazza az "Id", "AssetCode" (Eszközsám), "QrGuid" és "LastInventoryDate" (Utolsó leltár dátuma) mezőket.
- **Generikus CRUD motor:** Tetszőleges szerkezetű táblák adatait képes dinamikus szótár-alapú adatformátumban kezelni ("SELECT", "INSERT", "UPDATE", "DELETE").
- **QR-kód generálás és beolvasás:** Az eszközökhöz automatikusan egyedi GUID-t rendel a szerver, amelyből PNG formátumú QR-kód képet generál a QRCoder könyvtár segítségével. A QR-kód beolvasásakor a szerver megkeresi a tételt az összes táblában, és frissíti az utolsó leltározási időpontot ("LastInventoryDate = UTC NOW").
- **In-Memory & Lemezes Authentikáció ("auth.json"):** Szálbiztos, memóriában tartott, JSON fájlba szinkronizált felhasználó- és jogosultságkezelés BCrypt jelszó-hashinggal és granularis táblaszintű hozzáférés-szabályozással ("None", "Read", "ReadWrite").
- **Futásidejű külső konfigurációk:** A kapcsolati adatokat ("dbsettings.json"), a HTTPS tanúsítványokat ("certsettings.json"), valamint a QR-kódok alap URL-jét ("qrsettings.json") külön JSON fájlokból tölti be az alkalmazás, így a publisholt bináris fordítás nélkül, egyszerű szövegszerkesztővel konfigurálható.
- **Automatikus kezdeti inicializáció:** Első indításkor automatikusan létrehozza a MySQL adatbázist, valamint az alapértelmezett "admin" / "admin123" fiókot, ha még nem léteznek.

Felhasznált technológiák és NuGet csomagok

- **.NET 10.0 (ASP.NET Core Web API)** – Magas teljesítményű webes keretrendszer.
- **MySqlConnection (2.6.2)** – Teljesen aszinkron, nagysebességű natív ADO.NET illesztő MySQL / MariaDB adatbázisokhoz.
- **BCrypt.Net-Next (4.2.0)** – Biztonságos jelszó-hashing (automatic salt generálással).
- **QRCoder (1.7.0)** – QR-kód generálás és PNG formátumú renderelés.
- **Swashbuckle.AspNetCore (10.2.3)** – OpenAPI specifikáció és Swagger UI támogatás.

4. Projektstruktúra és Összetevők

- Controllers/
 - AuthController.cs Bejelentkezés, regisztráció, felhasználó- és jogkezelés
 - DataController.cs Generikus CRUD végpontok dinamikus táblákhoz
 - QrController.cs PNG formátumú QR-kód generálása QrGuid alapján
 - QrSettingsController.cs QR-kódokhoz tartozó szervercím kezelése
 - ScanController.cs QR-kód beolvasás és automatikus leltározás
 - TablesController.cs Dinamikus táblák és oszlopok létrehozása/törlése
- Data/
 - AuthConfigService.cs auth.json száلبiztos In-Memory & lemezes kezelése
 - IAuthConfigService.cs AuthConfigService interfész
 - IDbConnectionFactory.cs Adatbázis-kapcsolat gyár interfész
 - MySqlConnectionFactory.cs MySqlConnection nyitáért felelős osztály
 - ReservedColumnNames.cs Fix rendszeroszlopok gyűjteménye (Id, AssetCode, QrGuid, stb.)
 - SqlColumnType.cs Zárt SQL típus-fehérlista és regex érvényesítő
 - SqlIdentifier.cs SQL azonosító-érvényesítő és backtick quoting
 - StartupInitializer.cs Első indításkori adatbázis- és admin-létrehozás
- Models/
 - AuthModels.cs Felhasználók, jogosultságok, DTO-k és kérések
 - QrSettingsModels.cs QR szervercím beállításainak modelljei
 - TableSchemaModels.cs Dinamikus táblák és oszlopok definíciós modelljei
- Repositories/
 - DynamicTableRepository.cs MySQL sémalekérdezések, DDL/DML végrehajtás és Scan logika
 - IDynamicTableRepository.cs DynamicTableRepository interfész
 - IQrSettingsService.cs IQrSettingsService interfész
 - QrSettingsService.cs qrsettings.json olvasása és mentése
- certs/
 - iskolaraktar.pfx Önáláírt HTTPS Kestrel tanúsítvány
- appsettings.json Logolási és Kestrel port beállítások
- dbsettings.json MySQL kapcsolati string (külsőleg szerkeszthető)
- certsettings.json PFX tanúsítvány elérhetősége és jelszava
- qrsettings.json QR alapcím konfiguráció
- iskolaraktarBackend.csproj Cprojektfájl és függőségek
- Program.cs Alkalmazás belépési pont, DI és middleware csővezeték

5. Konfigurációs Fájlok és Környezeti Beállítások

Az alkalmazás futási viselkedését és külső függőségeit több JSON konfigurációs fájl határozza meg, amelyeket a [iskolaraktarBackend/Program.cs] (iskolaraktarBackend/Program.cs) tölt be indításkor.

5.1 "appsettings.json"

A Kestrel webservert figyelési portjait és a logolási szinteket állítja be.

5.2 "dbsettings.json"

A MySQL adatbázis-szerverhez való kapcsolódás adatait tartalmazza. Publikált binárisok esetén közvetlenül módosítható az adatbázis címe, felhasználója és jelszava.

5.3 "certsettings.json"

A HTTPS végpont által használt SSL/TLS tanúsítvány (PFX) elérési útját és jelszavát definiálja.

5.4 "qrsettings.json"

A nyomtatható QR-kódokba kódolt publikus frontend vagy szerver alap URL-jét tartalmazza.

5.5 "auth.json" (Automatikusan jön létre)

A rendszer felhasználóit, hashed jelszavaikat és granularis táblaszintű jogosultságait tárolja. Az iskolaraktarBackend/Data/AuthConfigService.cs kezeli.

6. Kezdeti Inicializáció és Indítási Folyamat

Az alkalmazás elindulásakor a iskolaraktarBackend/Program.cs-ben regisztrált iskolaraktarBackend/Data/StartupInitializer.cs fut le a kérések kiszolgálása előtt.

Az inicializációs lépések sorrendje:

1. Adatbázis ellenőrzés és létrehozás:

- Kiolvassa a "dbsettings.json"-ból a kapcsolati stringet.
- Kinyeri az adatbázis nevét (pl. "iskolaraktar"), majd ideiglenesen üres adatbázisnévvel csatlakozik a MySQL szerverhez.

2. Felhasználókezelő ellenőrzés és alapértelmezett admin:

- Ellenőrzi, hogy létezik-e az "auth.json" fájl.
- Ha nem létezik (első indítás), automatikusan inicializálja az "auth.json"-t, és létrehozza a bemutatóhoz/adminisztrációhoz használható alapértelmezett fiókot:
 - **Felhasználónév:** "admin"
 - **Jelszó:** "admin123" (BCrypt-tel hashelve)
 - **Jogkör:** "IsAdmin = true", full hozzáférés ("*" = ReadWrite).

7. Dinamikus Tábla- és Adatkezelő Motor

7.1 A dinamikus architektúra lényege

A hagyományos ORM-ekkel (mint pl. az EF Core) ellentétben az **Iskolaraktár** nem használ statikus Centitásosztályokat a raktári tárgyakhoz, mert a felhasználó a felületről bármikor új raktári kategóriákat hozhat létre vagy törölhet, illetve új mezőket adhat hozzájuk.

A iskolaraktarBackend/Repositories/DynamicTableRepository.cs a MySQL "INFORMATION_SCHEMA" tábláiból kérdezi le a struktúrákat, és az adatokat dinamikus "Dictionary<string, object?>" típusú rekordokként kezeli.

7.2 Rendszer által kezelt fix (fenntartott) oszlopok

Minden újonnan létrehozott dinamikus táblában a iskolaraktarBackend/Data/ReservedColumnNames.cs alapján automatikusan létrejön 4 fix oszlop:

Oszlopnév	Típus	Leírás
Id	INT AUTO_INCREMENT PRIMARY KEY	Egyedi rekordazonosító
AssetCode	VARCHAR(64) NOT NULL UNIQUE	Az eszköz egyedi, ember által is olvasható azonosító kódja
QrGuid	CHAR(36) NOT NULL UNIQUE	A szerver által beszúrásakor generált egyedi GUID, erre mutat a QR-kód
LastInventoryDate	DATETIME NULL	Utolsó sikeres QR-kódos leltározási szkennelés időpontja (UTC)

Ezeket a neveket a felhasználó nem adhatja meg egyedi oszlopként, és a szerver elutasítja, ha valaki felül szeretné írni a "QrGuid"-ot frissítéskor.

7.3 Biztonság és SQL Injection elleni védelem

Mivel a tábla- és oszlopneveket, valamint az SQL típusokat nem lehet hagyományos SQL paraméterként átadni a MySQL-nek, az alkalmazás szigorú kétlépcsős érvényesítést alkalmaz:

1. Azonosítók ellenőrzése (iskolaraktarBackend/Data/SqlIdentifier.cs):

- Minden tábla- és oszlopnévnek meg kell felelnie a "[A-Za-z_][A-Za-z0-9_]{0,63}" reguláris kifejezésnek.

- Az azonosítók idézőjelezése MySQL backtick-kel történik ("Quote()" metódus).

2. SQL típusok fehérlistája ([iskolaraktarBackend/Data/SqlColumnType.cs]

(iskolaraktarBackend/Data/SqlColumnType.cs)):

- Csak a kijelölt, biztonságos SQL típusok engedélyezettek a következő zárt reguláris kifejezés alapján:

(INT|BIGINT|SMALLINT|TINYINT|BOOLEAN|BOOL|TEXT|DATE|DATETIME|TIMESTAMP|DOUBLE|FLOAT|VARCHAR(\d{1,4})|CHAR(\d{1,4})|DECIMAL(\d{1,2},\d{1,2}))\$

8. Authentikáció és Jogosultságkezelés

Az autentikációt és a jogosultságokat a [iskolaraktarBackend/Data/AuthConfigService.cs] (iskolaraktarBackend/Data/AuthConfigService.cs) osztály kezeli.

8.1 Felhasználói modell és állapot

- **BCrypt hashing:** A jelszavak soha nem tárolódnak sima szöveggént.

- **Szálbiztosság:** A memóriabeli lista módosítása és lemezre írása "SemaphoreSlim" zárolás alatt történik.

8.2 Jogosultsági szintek ("AccessLevel")

- "None" (0): Nincs hozzáférés a táblához.

- "Read" (1): Csak adatok lekérdezése engedélyezett ("GET").

- "ReadWrite" (2): Teljes CRUD jogkör ("GET", "POST", "PUT", "DELETE").

8.3 Hozzáférés-szabályozási szabályok

- Ha "IsAdmin = true", a felhasználó korlátozás nélkül mindent elér.

- Egyébként a "permissions" szótárban lévő táblaspecifikus kulcs határozza meg a jogot.

- Ha az adott táblanév nem szerepel kifejezetten a szótárban, a wildcard ("*") kulcshoz tartozó jogkör az érvényes.

9. QR-Kód és Leltározó Rendszer

A leltározási munkafolyamat a szerver által előállított QR-kódokra és azok beolvasására épül.

9.1 QR-kód generálása

([iskolaraktarBackend/Controllers/QrController.cs]

(iskolaraktarBackend/Controllers/QrController.cs))

- Amikor egy új rekordot szúrunk be a "DataController"-en keresztül, a "DynamicTableRepository" automatikusan generál egy új "Guid"-ot a "QrGuid" oszlopba.
- A kért "GET /api/qr/{qrGuid}.png" végpont a "qrsettings.json"-ban konfigurált "BaseUrl" alapján összefűzi a nyilvános URL-t (pl. "https://localhost:7142/scan/a1b2c3d4-..."), majd a QRCode könyvtár segítségével PNG képpé alakítja azt és visszaadja HTTP válaszként ("image/png").

9.2 Leltár szkennelése

([iskolaraktarBackend/Controllers/ScanController.cs]

(iskolaraktarBackend/Controllers/ScanController.cs))

- Amikor a leltározó eszköz (pl. mobiltelefon vagy vonalkódozó) beolvassa a QR-kódot, a frontend egy "POST /api/scan/{qrGuid}" kérést küld a backendnek.
- A "DynamicTableRepository.ScanAsync" metódusa végigpásztázza az adatbázis összes létező tábláját, és megkeresi a megadott "QrGuid"-val rendelkező sort.
- Amikor megtalálja a tételt:
 1. Azonnal frissíti a "LastInventoryDate" mezőt az aktuális UTC időpontra ("UPDATE ... SET LastInventoryDate = NOW() WHERE QrGuid = ...").
 2. Visszaadja a megtalált tábla nevét és a frissített tétel teljes adatstruktúráját.

10. Részletes API Végpont Referencia

Swagger API dokumentáció van telepítve a backendbe. Onnan lehet megnézni melyik endpoint hogyan működik.

11. Telepítés, Publikálás és Üzemeltetés

11.1 Önálló (Self-Contained) Publikálás

A gyökérkönyvtárban lévő publish.sh (Linux) és publish.bat (Windows) scriptek segítségével mind a backend, mind a frontend automatikusan lefordítható önálló, single-file futtatható állományokká.

A fordítás eredménye a "publish/" mappába kerül:

- "publish/win-x64/backend/" – Windows x64 futtatható állomány ("iskolaraktarBackend.exe")
- "publish/linux-x64/backend/" – Linux x64 futtatható állomány ("iskolaraktarBackend")

A célgépeken **nem szükséges telepített .NET SDK vagy Runtime**, mert az önálló binárisok tartalmazzák a szükséges futtatókörnyezetet.

11.2 HTTPS és Tanúsítvány Kezelés

A publikált alkalmazás az [iskolaraktarBackend/certsettings.json] (iskolaraktarBackend/certsettings.json) fájlban megadott PFX tanúsítvánnyal indul el. Éles vagy belső hálózati környezetben a tanúsítvány bármikor lecserélhető saját vagy céges tanúsítványra a "certsettings.json" átírásával és a ".pfx" fájl kicserélésével.

12. Fejlesztési és Biztonsági Útmutató

1. Jelszavak és Titkok:

- A "dbsettings.json" és "certsettings.json" fájlok érzékeny adatokat (adatbázis jelszó, tanúsítvány jelszó) tartalmaznak.
- Éles üzemben javasolt a fájlokra megfelelő operációs rendszer szintű olvasási jogosultságokat beállítani.

2. CORS Beállítások:

- A iskolaraktarBackend/Program.cs-ben a CORS házirend engedélyezi a frontend elérését ("https://localhost:7142" és "http://localhost:5283"). Éles üzemben a domain nevét szükség esetén bővíteni kell.

3. Adatbázis Mentés:

- A teljes raktári adatbázis a MySQL szerveren található. Rendszeres "mysqldump" mentés javasolt az adatbázisról, valamint az "auth.json", "dbsettings.json", "qrsettings.json" konfigurációs fájlokról.